

CSCI 112

Programming with C

LECTURE: 9

J. L. Pach

OUTLINE:

- Loop statements
  - while
  - do-while
  - for
- break, continue

LOOP

statements

WHILE LOOP 1/2

```
while (expression1)
        statement1;
```

A `while` loop repeatedly executes `statement1` based on the logical outcome of expression1 as long as `expression1` evaluates to `true`.

```
int main()
{
  int i = 0;
  while ( i < 3 )
    printf("a value of i is: %d\n", i++);
}
```

OR

```
int main()
{
  int i = 0;
  while ( i < 3 )
  {
    printf("a value of i is: %d\n", i);
    i = i + 1; /* OR */ //i++;
  }
}
```

```
Result:
a value of i is: 0
a value of i is: 1
a value of i is: 2
```

WHILE LOOP 2/2

```
int main()
{
  int i = -1;
  while ( ++i < 3 )
    printf("a value of i is: %d\n", i);
}
```

OR

```
int main()
{
  int i = 0;
  while ( i < 3 )
    printf("a value of i is: %d\n", i++);
}
```

```
int main()
{
  int i = 0;
  while ( i < 3 )
  {
    printf("a value of i is: %d\n", i);
    i = i + 1;
  }
}
```

Result:

a value of i is: 0

a value of i is: 1

a value of i is: 2

for loop

```
for (expression1; expression2; expression3)
    statement1;
```

- A `for` loop repeatedly executes `statement1` based on the logical outcome of `expression2` as long as `expression2` evaluates to `true`.
- Most commonly `expression1` and `expression3` are assignments of function calls and `expression2` is a relational expression.
- Any of the three part can be omitted, although the semicolons must be remain. If `expression2` is not present, it is taken as permanently `true`.

```
int main()
{
  for (int i = 0; i < 4; i++ )
    printf("a value of i is: %d\n", i);
}
```

```
Result:
a value of i is: 0
a value of i is: 1
a value of i is: 2
```

THE FOR LOOP IS EQUIVALENT TO WHILE LOOP

```
expression1;
while (expression2)
{
        statement1;
        expression3;
}
```

```
for (expression1; expression2; expression3)
{
        statement1;
}
```

“INFINITY” LOOP

```
while (true)
{
        ;
}
```

```
for (;;)
{
        ;
}
```

DO-WHILE LOOP

```
do
  statement1;
while(expression1)
```

A `do-while` loop executes `statement1`, then checks the logical outcome of `expression1`. If `expression1` evaluates to `true`, the loop repeats; otherwise, the loop terminates.

```
int number;
do
{
    printf("Enter a positive number: ");
    scanf("%d", &number);
}
while (number <= 0);   /* <===== semicolon! */
printf("You entered: %d\n", number);
```

```
Result:
Enter a positive number: -9
Enter a positive number: 2
You entered: 2
```

SIMULATING A DO-WHILE LOOP WITH WHILE LOOP

```
int main()
{
  bool flag = true;
  while (flag)
  {
      printf("This code will execute at least once.\n");
      if (! (/* condition */) )
          flag = false;
  }
}
```

*In Python we don't have do_while.

Example

```
int main()
{
    int number;
    bool flag = true;
    while (flag)
    {
        printf("Enter a positive number: ");
        scanf("%d", &number);
        if (! (number <= 0) )
            flag = false;
    }
    printf("You entered: %d\n", number);
}
```

```
Result:
Enter a positive number: -9
Enter a positive number: 2
You entered: 2
```

BREAK & CONTINUE KEYWORDS 1/2

- `break` and `continue` offer distinct ways to control loop flow.
- `break` instantly terminates the loop it's in, while `continue` jumps to the next iteration.
- When nested, break affects only the immediate loop.
- `break` is also applicable in switch statements to exit a specific case

```c
int main()
{
  for (int i = 1; i <= 10; i++)
  {
    if (i == 5)
      break;          /* Breaks the loop      */
    printf("%d ", i); /* when i is equal to 5 */
  }
}
```

Result:

1 2 3 4

break & continue keywords 2/2

- break and continue offer distinct ways to control loop flow.
- break instantly terminates the loop it's in, while `continue jumps to the next iteration.`
- When nested, break affects only the immediate loop.
- break is also applicable in switch statements to exit a specific case

```c
int main()
{
  int i = 0;
  while (i < 10)
  {
    i++;
    if (i % 2 == 0)
      continue;       /* Skips the rest of the    */
    printf("%d ", i); /* iteration when i is even */
  }
}
```

```
Result:
1 3 5 7 9
```

goto statement in loops

```c
int main()
{
  bool bug = false;

  for (int i = 0; i < 5; i++)
  {
    for (int j = 0; j < 5; j++)
    {
      /* some code */
      /* ...       */
      if(bug)
        goto error;
    }
  }
  goto end;
  error:
    /* some code */
    /* ...       */
    printf("error found");
  end:
}
```

```c
int main()
{
  bool bug = false; int i = 0, j = 0;
  while ( i < 5)
  {
    while ( j < 5)
    {/* some code */
     /* ...       */
      if(bug)
        goto error;
      j++;
    }
    i++;
  }
  goto end;
  error:
    /* some code */
    /* ...       */
    printf("error found");
  end:
}
```

goto provides a convenient way to exit from nested blocks

SUMMARY OF THE DIFFERENCE BETWEEN FOR AND WHILE LOOPS

- The statement about using for for a known number of iterations and while for an unknown number is an excellent guideline, but it's not a strict rule. It's more of a best practice for writing clean, readable, and maintainable code.
- In most programming languages, these two loops are technically interchangeable. You can always write a for loop using a while loop (by manually handling the counter variable), and you can often force a while loop into a for structure, though it might look awkward.
- So, while it's possible to use them interchangeably, following the convention makes your code more understandable for yourself and for other developers. It's about choosing the right tool for the job to make your code's purpose immediately clear.

THANK

You